

Introduction

In today's lecture we discuss about subject of this course, its motivation and relation to the real world.

Subject of this course is to develop a rigorous, mathematical model of computation and computers.

Motivation ~~of~~ for this is to examine and understand the power and limitation of computation and computers.

As, we are interested in fundamental questions, such as what is computable and what is not in our model (possibly with some restrictions), our model should be as simple as possible. That is, we do away with all inessential aspects and keep only bare minimum constructs in our model.

Rigorous in the earlier description refers to the fact that model should be faithful and complete w.r.t. the real world notion of computation. A model being mathematical ensures that it is precise and unambiguous. It is worth emphasizing these characteristics of our models/formalisms by listing them below

- Simple
- Mathematical
- Rigorous

We may also consider another ~~prop~~ property of a model:

- Concrete

(a mathematical model may still be abstract...)

Exercise: A computational model that you all are familiar with is programming language C. Which of the above properties does this model satisfy?

Before discussing computers and computations further, it is meaningful to ask, what are we computing? For concreteness we assume that we are interested in ~~com~~ computing $\#$ fns $N \rightarrow N$.

That is we are considering a classical device which takes a number ($\in N$) as input and produces an answer $\in N$, as an output.

Is $N \rightarrow N$ too restrictive? What about fns which take more than one argument, even simple fns like addition and multiplication take two arguments.

It is easy to code a sequence of finitely many natural numbers in a single natural number. For example, consider numbers 1101 & 1110 ^{written} in binary notation. we encode them as 1101#1110, where $\# \in \{0,1\}$. Now each symbol can be encoded as follows:

$$\begin{array}{ll} 1 \rightarrow 11 & \# \rightarrow 01 \\ 0 \rightarrow 00 \end{array}$$

So, we can write above string 1101#1110 as

$$1111001101111100$$

this can be regarded as a single number (with binary rep as above). It is also easy to see how to recover two numbers from the above ~~string~~ single number. So $N^2 \rightarrow N$ can also be coded as $\#$ a fn $: N \rightarrow N$. Similarly $f: N^m \rightarrow N$ can be coded by another $f_n: N \rightarrow N$. So, $f: N \rightarrow N$ is not as restrictive as it may have seemed in the beginning.

Now let us consider another example. Suppose we are given a weighted graph $G = (V, E, w: E \rightarrow \mathbb{N})$ and $s, t \in V$. We need to compute the ~~dist~~ shortest path distance between s and t . Can this be modeled as $f_n: \mathbb{N} \rightarrow \mathbb{N}$.

Let us see how can we encode G as a number. First, we encode G as a string over a finite alphabet.

We can number vertices of G as $1, \dots, n$ for some $n \in \mathbb{N}$. Each can be written as a triple (i, j, θ) , where $i, j, \theta \in \mathbb{N}$ and $\theta = w(i, j)$. So the whole graph G can be written as

$$(n, ((i_1, j_1, \theta_1), (i_2, j_2, \theta_2), \dots, (i_m, j_m, \theta_m)), i, j)$$

Where i, j are ~~set~~ numbers corresponding to vertices s and t respectively. Note that $n, i_k, j_k, \theta_k, i, j$ all can be written as binary strings, that is over alphabet $\{0, 1\}$. Therefore, whole description ~~of~~ of G ~~above~~ above is a string over alphabet $\{0, 1, '(', ')', ', '\}$ of size ^{five} ~~five~~. Each element of this alphabet may be coded as a string ~~in~~ in $\{0, 1\}^3$. So the whole graph encoding can be written as a binary number (involving '0' and '1's only). The decoding f_n is also quite easy. ~~As~~ ~~As~~ G may be coded as a number, the shortest distance problem over graphs can also be ~~computed~~ ~~as~~ coded as ~~for~~ computing a $f_n: \mathbb{N} \rightarrow \mathbb{N}$.

Exercise! Given $m, n, p \in \mathbb{N}$ and two matrices $A_{m \times n}$ and $B_{n \times p}$ over N , Consider the problem of computing a $C_{m \times p}$ s.t. $C = A \cdot B$. Formulate this as a problem of computing a fn $f: N \rightarrow N$.

This should convince the reader that almost all ^{Computational} common problems can be represented as computing a fn $f: N \rightarrow N$.

We shall, in fact represent problems ~~even~~ as an even more convenient formalism. For that, we give some notations and definitions below.

Defn (Alphabet): An alphabet is a finite set of symbols.

Examples:

$\{0, 1, \dots, 9\}$

$\{a, b, \dots, z\}$

$\{\#, \$, \% \}$

$\{\text{All ascii characters}\}$

A string is a finite juxtaposition of symbols.

Example

1190 is a string of length 4. (over alphabet $\{0, 1, \dots, 9\}$)

A word ~~is a finite~~ over alphabet Σ is a string consisting of symbols of Σ .

Notation: Σ is used to denote alphabet.

Defn $\Sigma^* = \{w \mid w \text{ is a (finite) word over } \Sigma\}$

Example $\{0,1\}^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, 001, \dots\}$

' ϵ ' is the empty string (with zero symbols).

Defn $\Sigma^i = \underbrace{\Sigma \times \Sigma \times \dots \times \Sigma}_{i \text{ times}}, \text{ for } i > 0$

$$\Sigma^0 = \{\epsilon\}$$

$$\Sigma^+ = \bigcup_{i \geq 1} \Sigma^i, \quad \text{***}$$

It may be observed that $\Sigma^+ = \Sigma^* - \{\epsilon\}$

$$\text{or } \Sigma^* = \bigcup_{i \geq 0} \Sigma^i.$$

Defn A language 'L' over alphabet Σ is a subset of Σ^* .
 $L \subseteq \Sigma^*$.

Examples 1. $\{0^i 1^j \mid i, j \in \mathbb{N}\}$ is a language over ~~the~~ alphabet $\{0, 1\}$.

2. $\{w \in \{a, b, c\}^* \mid w \text{ has equal no. of } a's, b's \text{ and } c's\}$
is a language over $\{a, b, c\}$.

Many of the models that we study have a simpler output structure than natural numbers. They output either 0 or 1. These may be identified with two states 'reject' and 'accept' respectively.

So, these models either accept or reject their input. The input itself is concrete, as an element of Σ^* (as opposed to an abstract entity like natural number). There may be thought of as functions in $\Sigma^* \rightarrow \{0, 1\}$, or equivalently subsets of Σ^* . This is nothing but a language.

Our models recognize a language over Σ . They may also be called language acceptors or ~~or~~ language recognizers.

We have seen above that a ~~problem~~ computational problem may be coded as a function $f_n : N \rightarrow N$. We now argue that a f_n may be reasonably be coded as a language.

Let $f : N \rightarrow N$. We define a language L_f over alphabet $\{0, 1, \#\}$ as follows.

$$L_f = \{ u \# v \mid \exists n \in N [\begin{array}{l} u \text{ is the binary rep. of } n, \\ v \text{ is a prefix of binary} \\ \text{representation of } f(n) \end{array}] \}$$

Example:

$$n = 4, \quad f(4) = 5$$

then

$$100\#1, \quad 100\#10, \quad 100\#101 \in L_f$$

Value of f at 4 can be ~~not~~ recovered from L_f by checking membership of increasing prefixes of answers for $f(4)$. [This requires ~~only~~ at most $2 \cdot f(4)$ membership checks in L_f]

Exercise:

In an earlier example, we considered the problem of computing shortest distance between two vertices in a graph. This was formulated as a fn ~~from~~ $N \rightarrow N$. Suppose we are interested in not just the shortest distance, but also the shortest path between the two vertices. What is a good way to represent this as a ~~fn~~ fn? ~~as a language~~ As a language? How many membership queries one may need (in terms of parameters of the graph) to the language in order to find the path?

we now have mathematical counterparts of notions like problems, expressive power of a model.

problem : Language (over finite alphabet)

Expressive power of a model : Class of languages recognized by it.

In rest of the course, we see various classes of languages, like
regular languages
context free languages
~~rec~~ recursive languages
etc.

We shall define models, recognizing these classes of languages and study various properties of these models.